

PA 1: Implementing xv6 User Commands and System Calls

Please replace red text with your report text and any tables or figures, names of any accompanying files, etc. Remember to commit all the files for your lab submission, to put the URL for your private xv6 repo in the Teams assignment, to submit the Teams assignment, and to give the instructor and TA access to your repo.

Task 1. (5 pts) Implement the uptime utility

User/uptime.c is in the user directory of the pa1 branch.

Results from running `$ uptime`:

```
$ ls
.          1 1 1024
..         1 1 1024
README    2 2 2441
cat        2 3 36808
echo       2 4 35640
forktest  2 5 17560
grep       2 6 40248
init       2 7 36096
kill       2 8 35504
ln         2 9 35392
ls         2 10 38944
mkdir     2 11 35624
rm         2 12 35616
sh         2 13 58688
stressfs  2 14 36488
usertests 2 15 202160
grind      2 16 51872
wc         2 17 37704
zombie    2 18 34976
logstress  2 19 37560
forphan   2 20 36432
dorphan   2 21 35888
sync      2 22 35040
uptime    2 23 35176
time1     2 24 36136
console   3 25 0
$ uptime
up 27862 clock ticks
$ uptime
up 27928 clock ticks
$
```

Task 2. (10 pts) Implement a time1 command that reports elapsed time.

Time1.c is in the user directory in the pa1-syscalls branch. Time was added to the UPROGS in the Makefile.

Results of `$time1`:

```
init: starting sh
$ time1 matmul
Time: 6 ticks
Elapsed time: 6 ticks
$ time1 matmul & time1 matmul &
$ Time: 11 ticks
Elapsed time: 12 ticks
Time: 10 ticks
Elapsed time: 11 ticks
time1 sleep 10
Elapsed time: 10 ticks
$
```

Task 3. (10 pts) Keep track of how much cputime a process has used.

Changes:

- kernel/proc.h – Added an “int cputime;” field to “struct proc” to record the amount of timer ticks a process has consumed while running.
- Kernel/proc.c – In allocproc(), initialized p->cputime = 0; when a new process is allocated so every new process starts with a new counter.
- Kernel/trap.c – in usertrap(), incremented p->cputime inside the existing timer interrupt check, right before the call “yield()”. Made the same changes in kerneltrap() , using “myproc() > cputime++”.

Purpose:

- Change 1 - The cputime is the actual counter now. It needed to live in struct proc so every process has its own independent value.
- Change 2 – Initializing it to 0 in allocproc() guarantees a fresh process never inherits a stale count from an earlier process that occupied the same slot.
- xv6 uses “which_dev == 2” to indicate that a trap was caused by the timer, which also calls “yield()”. Incrementing cputime at that exact point connects the counter with "the process just used up one tick of CPU time." kerneltrap() needed the same change because a process can be canceled while running in kernel mode, not just user mode.

Task 4. (10 pts) Implement a wait2() system call that waits for a child to exit and returns the child’s status and rusage.

Changes:

- Kernel/pstat.h – Defined struct rusage {uint cputime; }; - the resource usage struct returned to the caller.
- User/user.h – Added forward declaration struct rusage; and the prototype int wait2(int *, struct rusage*);
- User/usys.pl – Added <entry(“wait2”);> so the build generates the user-space assembly stub for the new syscall.
- Kernel/syscall.h – Added #define SYS_wait2 <N> with the next available syscall number.
- Kernel/syscall.c – Added extern uint64 sys_wait2(void); and registered it in the syscalls[] table as [SYS_wait2] sys_wait2.
- Kernel/defs.h – Added the prototype int kwait2(uint64, uint64); so kwait2() is visible outside of proc.c.
- Kernel/proc.c – Added #include “pstat.h”. Implemented kwait2() by copying wait()’s logic and extending it to also copy a struct rusage out to the caller’s second argument.
- Kernel/sysproc.c – Implemented sys_wait2() , which pulls the two user space pointer arguments with argaddr() and calls kwait2().

Purpose:

- Pstat.h defines the returned data in a separate header, allowing kernel and user code to share the structure definition.
- A user application can call wait2() like any other library function thanks to the user.h/usys.pl changes; usys.pl is what creates the low-level trap/ecall stub.

- `kwait2()` mirrors `wait()`'s child-reaping loop exactly, with one addition, before calling `freeproc()` (which resets the child's `proc` struct, including `cputime`), it copies the child's `cputime` out to the caller's `rusage` pointer via `copyout()`. This ordering matters — copying after `freeproc()` would return zero, since the field gets reset.

Task 5. (10 pts) Implement a `time` command that runs the command given to it as an argument and outputs elapsed time, CPU time, and %CPU used.

`User/time.c` is in the `user` directory. `time` was added to `UPROGS` in the `Makefile`.

Results from running `time`:

```
xv6 kernel is booting
init: starting sh
$ matmul
Time: 6 ticks
$ time matmul
exec time failed
$ time1 matmul
Time: 6 ticks
elapsed time: 6 ticks, cpu time: 6 ticks, 100% CPU
$ time1 matmul &
Time: 11 ticks
elapsed time: 12 ticks, cpu time: 6 ticks, 50% CPU
$ time1 sleep 10
elapsed time: 10 ticks, cpu time: 0 ticks, 0% CPU
$
```

Task 6. (5 pts) Summarize what you learned from doing this assignment. Describe any difficulties you faced and if/how you overcame them.

The assignment showed me how a `syscall` gets from a user space into the kernel and then back to the user. I didn't truly understand how the combinations of `fork()`, `exec()`, and `wait()` worked but this process showed me the chain of steps that take place. Additionally I learned how `xv6` tracks time. Ticks aren't just wall clock time but they are a count of timer interrupts, incremented once per hardware timer firing. Tying `cputime` tracking to the `yield()` call in `usertrap()/kerneltrap()` showed me the duration a process ran and how long a process has been alive are two completely different things. A difficulty I did encounter were confusing "elapsed time" and "CPU time" until I tested the processes and discovered the difference between wall ticks and CPU Ticks. I did also face small issues such as bug fixes where they didn't show up as blatant compile errors such as confusing `printf` and `fprintf`. I simply overcame these difficulties by proceeding to analyze my code and understanding exactly what each function was doing and the importance of it. By trial and error, it helped me realize my mistakes, correct them, and prevent them from appearing in future occurrences.

Extra Credit (5 points). Discuss limitations of our `time` command. (Hint: For one limitation, consider what would happen if the command that is being timed forks child processes).

A limitation of our `time` command is that it doesn't account for CPU time used by grandchildren. The `cputime` only tracks ticks charged to the exact process that's running when a timer interrupt fires. For example, if `matmul` itself calls `fork()` and does work in a child process, that child gets its own `cputimer` counter and its never rolled up to the parent's total. So when `time` calls `wait2()`, it only gets the immediate child's `cputime`, not any CPU time consumed by that child's own descendant.

Another limitation would be no accounting for time spent already exited or killed abnormally. If a timed command is killed, the time command still calls wait2() and gets some usage back but there is no clear distinction between “ran till completion normally” and “was terminated early”. Therefore, the ticks reported could be misleadingly low without an indication that the command actually didn’t actually finish its work.

Resources:

Anthropic Claude: Claude AI was used for debugging and assistance with understanding of processes and functions within the given files. Examples of some prompts used were “Analyze the following code and provide suggestions and feedback on any possible or current issues” or “Explain the following code and how it coexists and functions with the other following files or methods.” Suggestions used by Claude were considered and used for the completion of the project when an issue resulting in a substantial amount of time being used to resolve it.